TRABAJO FIN DE MÁSTER

Máster en Ingeniería del Software: Cloud, Datos y Gestión TI.

Configuration bugs classification using LLMs and encoders,

a practical approach

Realizado por
Noelia López Durán

Dirigido por
David Benavides Cuevas
David Romero Organvídez

June 2025
Academic Year 2024/2025

ABSTRACT

Configuration-related bugs in software development are reported to be one of the main sources of problems in different domains and are especially relevant in software product lines. Classifying configuration bugs is difficult, since most of the bug tracking systems use descriptions written in natural language that have to be interpreted to categorize them. Some projects use labels to classify bugs, but some others do not. In this paper, we address the challenge of automatically classifying bug reports as either configuration-related or non-configuration-related by leveraging the capabilities of large language models (LLMs). We conducted experiments involving zero-shot classification using general-purpose LLMs, as well as fine-tuning a smaller encoder-based model. These approaches were evaluated on two different datasets: first, a dataset consisting of 1,331 bug reports from a web-based software product line, manually labelled by us; and second, a refined version of an existing dataset from the literature, improved through additional data curation. The results are promising, with an F1 score of 0.81 using a combination of both datasets. Furthermore, we applied the fine-tuned encoder-based model to estimate the proportion of configuration-related bugs within a large, previously unseen dataset extracted from various highly configurable Eclipse projects, which contains over 300k bug reports. Our analysis yielded an estimated proportion of 36% for configuration-related bugs. To the best of our knowledge, this is the first work using LLMs to classify configuration bugs. Our results open a new perspective on configuration bugs classification and pave the way to investigate further in this direction.

RESUMEN

Los errores relacionados con la configuración en el desarrollo de software se reconocen como una de las principales fuentes de problemas en distintos dominios, siendo especialmente relevantes en las líneas de productos software. Su clasificación resulta compleja, dado que la mayoría de los sistemas de seguimiento de incidencias utilizan descripciones en lenguaje natural que deben ser interpretadas para poder categorizar adecuadamente los informes. Mientras algunos proyectos incorporan etiquetas para facilitar esta tarea, otros carecen de dicha sistematización. En este trabajo se aborda el reto de clasificar automáticamente los informes de errores como relacionados o no con la configuración, aprovechando las capacidades de los modelos de lenguaje de gran tamaño (LLMs). Para ello, se realizaron

experimentos utilizando técnicas de clasificación zero-shot con LLMs de propósito general, así como el ajuste fino de un modelo *encoder* de menor tamaño. Estas estrategias se evaluaron sobre dos conjuntos de datos distintos: el primero, compuesto por 1.331 informes de errores de una plataforma web basada en una línea de productos software, etiquetados manualmente en este trabajo; y el segundo, una versión refinada de un conjunto de datos previamente existente en la literatura, mejorado mediante un proceso de sampling y curación de los datos. Los resultados son prometedores, alcanzando un F1-score de 0,81 mediante la combinación de ambos conjuntos de datos. Además, se aplicó el modelo ajustado para estimar la proporción de errores relacionados con la configuración en un gran conjunto de datos no etiquetado, compuesto por más de 300.000 informes extraídos de diversos proyectos altamente configurables del ecosistema Eclipse. El modelo estimó que un 36 % de dichos informes correspondían a errores de configuración. Hasta donde alcanza nuestro conocimiento, este es el primer trabajo que emplea LLMs para esta tarea. Los resultados obtenidos abren una nueva perspectiva en la clasificación automática de errores de configuración y sientan las bases para futuras investigaciones en esta línea.

ACKNOWLEDGMENTS

To my family, whose support and love have carried me through every step of this journey. To my friends, who stood by me with encouragement, laughter, and shared moments that made the path lighter. To all the members of the Diverso Lab, who gave me the opportunity to work with them, instructed me and made this work possible.

With all my heart, thank you for being part of this chapter in my life and making this work possible.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

Part I

PREFACE

1 INTRODUCTION

Software systems are increasingly built as configurable platforms, enabling adaptation to diverse deployment environments, feature requirements, and performance needs. In such contexts, configuration-related bugs (those arising from misconfigured parameters, feature combinations, or infrastructure setups)have become a persistent and critical issue [7]. Empirical studies report that up to 31% of high-severity defects in both commercial and open-source systems are attributed to configuration problems [27].

These bugs are particularly prevalent in Software Product Lines (SPLs) [24], a development paradigm focused on the systematic reuse of shared assets to produce a family of related software products. SPLs enable efficient customization by allowing developers to configure features from a common codebase to meet specific requirements. While this approach promotes scalability and adaptability, it also introduces considerable complexity in managing configurations, often leading to subtle and hard-to-detect configuration-related bugs. Despite their significance, identifying and classifying configuration-related bugs remains a challenge. Bug reports are typically written in natural language, making it difficult to extract structured insights or to automate the triaging process.

Current tools and methods rely heavily on manual labelling, outdated taxonomies, or domain-specific knowledge that limits generalizability. Moreover, existing datasets often lack a clear definition of what constitutes a configuration bug, leading to label ambiguity and class overlap. This lack of standardization affects both reproducibility and the development of reliable, scalable classification models.

The main goal of this work is to explore and evaluate modern Natural Language Processing (NLP) techniques, specifically Large Language Models (LLMs) and transformer-based encoders, for the classification of configuration-related bug reports. This task is approached not only from a practical perspective, aiming to improve software maintenance and triaging processes, but also from a scientific standpoint, contributing new methods and empirical evidence to the research community.

To guide this work, the following specific objectives were defined:

- **Conceptual clarification**: To advance the theoretical understanding of what constitutes a configuration-related bug, based on existing literature and taxonomies such as those proposed by Siegmund et al [19].

- **Application of modern NLP models**: To investigate the feasibility of using general-purpose LLMs in a zero-shot setting for configuration bug classification, and to compare their performance with a fine-tuned transformer-based encoder trained on domain-specific data.
- **Empirical evaluation of methodologies**: To systematically compare prompt-based zero-shot classification with supervised learning approaches, using standard metrics such as precision, recall, and F1-score.
- **Dataset preparation**: To prepare a high-quality dataset specifically targeting configuration bugs, addressing the scarcity and inconsistency of existing resources.
- **Open science and reproducibility**: To ensure all code, data, and experimental results are published openly, promoting transparency and enabling future research in this area.
- **Real-world relevance**: To apply the best-performing model to a large-scale industrial dataset, in order to estimate the prevalence of configuration-related bugs in practice.

These objectives are designed to address both foundational gaps in the literature and practical needs in software engineering workflows, with a strong emphasis on scalability, reproducibility, and methodological rigour.

To address the classification of configuration-related bugs, this work proposes a dual methodological approach that leverages recent advances in NLP: zero-shot classification using a general-purpose Large Language Model (LLM), and supervised fine-tuning of a smaller transformer-based encoder.

The overall methodology is structured into three main phases:

1. **Dataset construction and prompt refinement**: Recognizing the limitations of existing datasets, we developed and manually labelled a dataset and curated an already existing one with overlapping issues. The first consists of 1,331 bug reports from the UVLHub platform, and the second is a refined version of the NABATS dataset [5], re-labelled to avoid ambiguous class definitions. These datasets not only serve for evaluation but also provide a foundation for supervised learning.
2. **Zero-shot classification using LLMs**: In this phase, a general-purpose language model, specifically, Gemini 2.0-flash, was queried using carefully engineered prompts. These prompts define the concept of configuration and request the model to classify a bug report as configuration-related or not, without requiring prior task-specific training. Several iterations of prompt design were conducted to improve the model's ability to distinguish relevant features.

3. **Supervised classification using fine-tuned models**: Lastly, a smaller encoder-based model, ModernBERT, was fine-tuned on the curated datasets. This phase aimed to assess whether domain-specific training could yield better results than general-purpose zero-shot approaches. The model was trained and validated using an 80/20 split, with performance evaluated via macro-averaged F1-score and class-specific metrics.

The final component of the approach consists of applying the best-performing model to a large-scale, real-world dataset with 301,465 bug reports from various Eclipse projects to produce a first estimation of the prevalence of configuration-related bugs in industrial settings. This step not only validates the model's scalability but also provides the first empirical insight into the frequency of configuration bugs in practice.

The evaluation of both zero-shot and fine-tuned approaches provided clear evidence of the effectiveness and limitations of each methodology. Zero-shot classification using Gemini 2.0-flash offered a practical baseline, showing that general-purpose LLMs can classify configuration-related bugs without prior training, provided that the prompts are carefully crafted and the input data is clean. However, the performance of this approach was inconsistent across datasets, particularly in the presence of label noise or ambiguous class definitions.

In contrast, the supervised fine-tuning of ModernBERT consistently outperformed the zero-shot model across all datasets. The model demonstrated strong classification capabilities, achieving a macro F1-score of 0.812 when trained on a combined dataset.

These results support several important conclusions. First, large language models, even in zero-shot settings, can provide a viable baseline for bug classification tasks. However, their performance is strongly tied to prompt quality and the consistency of the input data.

Second, fine-tuning a smaller encoder on high-quality, task-specific data significantly improves performance, demonstrating that domain adaptation remains a highly effective strategy, particularly in software engineering tasks where annotated data is limited but critical.

This work also demonstrates the importance of well-curated datasets, methodological transparency, and reproducibility. By making all artifacts (code, data, and models) publicly available, the study aligns with open science principles and fosters further research in this area.

Looking ahead, several directions could extend the contributions of this work. One promising opportunity involves evaluating alternative language models (both more recent LLMs and lightweight encoder architectures) to better understand the trade-offs between performance and computational cost. Expanding the dataset to include a broader variety of projects and domains would enhance generalizability, while semi-automated labelling strategies could help scale dataset creation with-

out sacrificing quality. Additionally, improving prompt robustness in zero-shot settings and exploring *human-in-the-loop* approaches may lead to more reliable and practical integration of these models into real-world software development workflows.

Part II

BACKGROUND

2 CONCEPTUAL REVISION

In this chapter, we conduct a concept review to understand the state of the art in the domain of classification of bug reports (specifically configuration bug reports) and help establish the context of our research. To this end, we group the main concepts into different sections and subsections.

2.1 INTRODUCTION

In order to develop reliable automated classification methods for configuration-related bug reports, it is essential to thoroughly understand the key concepts and theoretical foundations underlying this research. These include the nature and scope of configuration in software systems, the role of mining software repositories (MSR) as a research discipline and data source, the evolution of classification techniques (particularly in natural language processing (NLP) contexts) and finally, the methodologies employed to evaluate the performance of such classification systems.

2.2 CONFIGURATION

In software engineering, the term "configuration" faces a multi-faceted nature given its broad scope across domains[19]. This complexity is further amplified by the significant research attention and industrial interest it has attracted[18], as well as by the predominantly context-dependent manner in which it is often employed in literature, leaving it definition to be interpreted by the reader.

To bring clarity, Siegmund et al. [19] conceptualize configuration through eight dimensions that reflect its diverse applications. The dimensions that describe what configuration means are:

- **Stakeholder** - Identifies everyone who deals with the configuration, such as developers, administrators, or end-users. Each stakeholder may have different goals, levels of expertise, and access to configuration options.
- **Configuration Type** - Distinguishes between domain-specific and technical configuration. Domain-specific configuration enables customization of software functionality for different users or purposes. Technical configuration comprises infrastructure configuration and development configuration, as

the kind of tools, configuration artifacts, and configuration effort differ substantially.

- **Binding Time** - It refers to the event of binding a configuration option to a certain value. The binding time of an option varies largely, depending on build time, deployment time, to load- and run time.
- **Configuration Artifact** - Describes where the configurations is distributed since every artifacts exhibit their own structure, syntax, and semantic.
- **Stage** - Configuration happens in different stages of the development process, such as development or testing. Each stage usually describes a different infrastructure environment of the running system, variables, and available resources. Additionally, it is often strongly connected to a stage in the CI/CD process, in which a software system is deployed and executed.
- **Life Cycle** - Describes the diverse aspects of configuration options from creation and maintenance over binding to deprecation, that is, all lifetime phases.
- **Intent** - Reflects the purpose or motivation behind the configuration, such as optimizing performance, enabling specific features, or adapting to a particular environment.
- **Complexity** - Relates to the scope and difficulty involved in performing a configuration, which may stem from the number of options, dependencies between options and number of dependencies.

To bring clarity, Siegmund et al. [19] conceptualize configuration through eight dimensions that reflect its diverse applications. However, due to this multidimensional scope, the term configuration often lacks exclusivity and may overlap with other categories, such as performance, security, or functionality. A bug, for instance, might be primarily classified under one of these categories but still originate from misconfigurations. This ambiguity poses challenges for precise classification and highlights the importance of contextual understanding when addressing configuration in software systems.

2.3 SOFTWARE PRODUCT LINES

As described by J. White et al [24], "Software Product-Lines (SPLs) are a technique for creating software applications composed from reusable parts that can be retargeted for different requirement sets". These software systems and applications can be represented with UVL (Universal Variability Language) [2] using feature models (as represented in Figure 1).

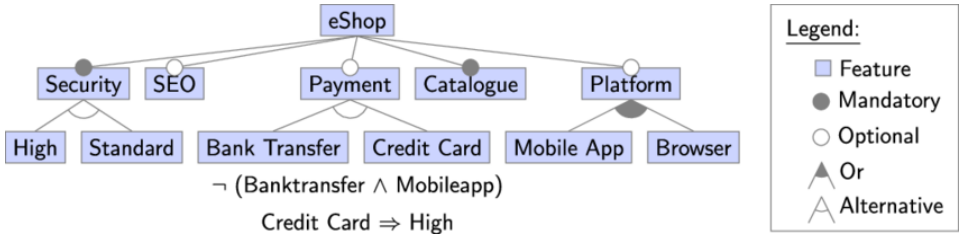

Figure 1: Representation of the feature model of a product using an SPL approach. (David Benavides et al. [2])

SPLs aim to promote systematic reuse and manage large-scale software variability by organizing systems as families of related products rather than developing each product independently.

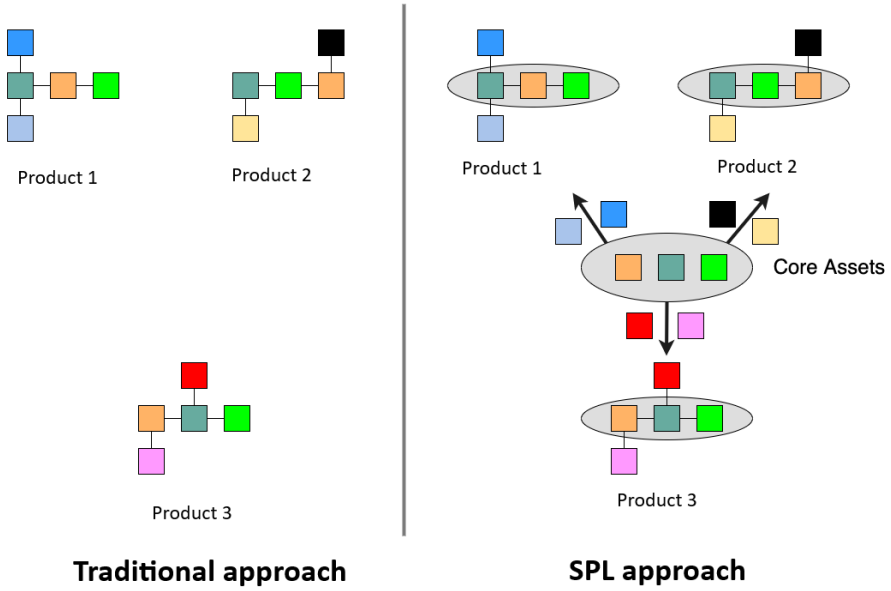

Figure 2: Traditional approach vs SPL approach about the construction of different and personalized products (obtained from DiversoLab's presentation material)

As illustrated in Figure 2, the SPL approach enables the efficient derivation of multiple tailored products from a shared set of core assets. This contrasts with the traditional approach, where each product is developed separately, leading to redundancy and increased maintenance overhead.

While the addition of configurable options enables benefits such as flexibility, personalization, and cost-effective customization, it also introduces **variability** (the range of possible combinations and feature interactions that a system has to manage in order to work correctly) which can be difficult to manage. This complexity

often results in increased testing demands, non-trivial integration workflows, and higher susceptibility to configuration-related bugs [7].

2.4 MINING SOFTWARE REPOSITORY(MSR)

Mining Software Repositories (MSR)[13], is a field dedicated to extracting and analysing data from software artifacts such as version control systems, issue trackers, and mailing lists. MSR enables empirical studies of software development practices and facilitates the application of machine learning models to real-world data [15].

Among the many applications of MSR, one key area has been the analysis of metadata. This area uses the metadata stored from a repository(e.g application logs, bug reports from Issue Tracking Systems(ITS), etc...). Early efforts in this direction leveraged textual similarity measures to group or classify bug reports based on their descriptions, summaries, or comments [22] to determine the time to fix a bug reported. These approaches extract textual features from bug reports collected using MSR pipelines, and then apply similarity-based metrics to infer categories or detect duplicates. For instance, two bug reports with similar descriptions may indicate a shared root cause or similar resolution time. While effective to some extent, these techniques often rely heavily on surface-level text patterns and may struggle to capture deeper semantic relationships without more advanced natural language understanding.

2.5 CLASSIFICATION

Classification is a fundamental task in machine learning and data analysis that involves assigning predefined labels or categories to data instances based on their features. The goal of a classification model is to learn a decision function that can generalize to unseen instances by predicting their class based on the learned patterns.

Historically, bug classification has been approached using traditional machine learning algorithms and natural language processing (NLP) techniques[14]. Commonly used models include Naive Bayes, Support Vector Machines (SVM), Logistic Regression, Decision Trees, and Random Forests[5, 20, 23], often combined with standard NLP feature representations such as bag-of-words, TF-IDF, and word embeddings[8, 20]. These models typically rely on handcrafted features extracted from textual descriptions, requiring extensive preprocessing steps such as stopword removal, tokenization, and frequency matrix construction[22]. In some cases, industrial tools such as SAS Text Miner have also been used to apply term-by-document transformations for bug report classification[9]. With the advent of deep learning, sequence models such as Long Short-Term Memory networks (LSTMs)

and convolutional neural networks (CNNs) were applied to bug report texts[8], enabling better semantic understanding by learning directly from word embeddings. However, these models often required large amounts of annotated data and computational resources, and still lacked generalization between domains or tasks.

As part of classification, zero-shot classification proposes a technique where models perform categorization tasks without being explicitly trained on labelled examples from the target domain [3, 16]. In a zero-shot setting, the classifier is guided by a textual description of the classification task, including definitions of the target labels. This prompt-based approach allows the model to apply its general knowledge to previously unseen data and categories. While zero-shot classification offers considerable flexibility and eliminates the need for labelled datasets, its performance can be sensitive to prompt design and model biases.

More recently, transformer-based architectures have emerged as powerful tools in NLP, offering promising results [6, 11, 12]. Pre-trained language models such as BERT [1] or RoBERTa [2] and their derivatives have been fine-tuned for various downstream classification tasks, including those in software engineering. Fine-tuning these models on domain-specific datasets allows them to learn task-relevant patterns with relatively limited data, offering improved accuracy and generalization [4].

These models are based on the transformer architecture introduced by Vaswani et al. [21], which relies on self-attention mechanisms to process the entire input sequence in parallel. Unlike traditional recurrent models, transformers can capture long-range dependencies more efficiently and scale well with large corpora.

2.6 PERFORMANCE METRICS

To assess model effectiveness, we use standard supervised learning metrics: precision, recall, and F1-score. Precision reflects the accuracy of positive predictions for each class, while recall quantifies, for each class, the proportion of true positive instances that were correctly identified by the model. The F1-score balances both, offering a harmonic mean of precision and recall. When dealing with class imbalance, the macro-averaged F1-score allows the evaluation with equally weighs performance across classes, ensuring a fair evaluation of all classes.

Accuracy, although commonly used, is intentionally omitted as a primary metric due to its sensitivity to class imbalance. In datasets where one class dominates, a model may achieve high accuracy by predominantly predicting the majority class, even if it fails to detect the minority class altogether. This is particularly problematic in tasks like configuration bug detection, where configuration-related reports may constitute a small fraction of the overall dataset. Therefore, macro-

1 https://huggingface.co/docs/transformers/model_doc/bert
2 https://huggingface.co/docs/transformers/model_doc/roberta

averaged metrics are more informative as they treat all classes equally, highlighting model robustness beyond superficial correctness.

Beyond overall performance, it is also important to examine how the model behaves across individual classes. Reporting class-specific F1-scores enables a more granular understanding of how well the model distinguishes between classes. For example, a model may achieve strong overall scores while still underperforming on the minority class. In such cases, taking the classification of configuration bugs as an example, the classifier might exhibit high recall but low precision, indicating that it identifies many configuration-related bugs but also produces a high number of false positives.

Part III

CONTRIBUTION

3 MOTIVATION

In this section, present the motivations that underpin this research by highlighting the relevance of configuration-related bugs in modern software systems. We identify key limitations in existing approaches, such as the lack of robust datasets and the limited use of advanced NLP techniques, and analyse the state of art and related work on this topic.

3.1 INTRODUCTION

Configuration-related bugs are recognized as a major source of software failures [25], particularly within highly configurable systems such as Software Product Lines (SPLs). These bugs often arise from incorrect settings, misused parameters, or unintended interactions among configuration options. According to Yin et al. [27], configuration issues account for up to 31% of high-severity bugs in both commercial and open-source software systems.

Despite their impact, identifying and classifying configuration bugs remains challenging. Bug reports are typically written in unstructured natural language, lacking consistent taxonomies or labelling practices. This leads to difficulties in automating bug triaging, reproducing findings, and developing targeted resolution strategies. The scarcity of annotated datasets and the lack of robust classification pipelines highlight a pressing need for improved methods in this domain.

This work aims to address this challenge by leveraging advances in Natural Language Processing (NLP), particularly through the use of Large Language Models (LLMs) and fine-tuned transformer-based encoders. These models offer new capabilities for understanding complex textual content and generalizing across domains, making them suitable candidates for the classification of configuration-related bug reports.

3.2 RELATED WORK

Several studies have addressed the classification of bug reports using machine learning and natural language processing techniques. However, only a few have specifically focused on configuration-related bugs or leveraged recent advancements in large language models (LLMs). Table 1 provides a comparative overview of relevant works according to various dimensions such as focus on configuration, dataset availability, model types, and reported performance.

Catolino et al. [5] proposed a taxonomy for bug classification and applied several classical supervised machine learning techniques (SVM, RF, NB, LR) on a dataset of 1,280 bug reports. Although their dataset is public and well-structured, the work does not focus on configuration bugs specifically, and the reported F1-score for configuration-related bugs is relatively modest (~0.64). In addition, as we discussed on Section 2.2, the term of configuration-related bugs is often non-exclusive and overlapping with other categories, which may introduce ambiguity in classification tasks that rely on mutually exclusive labels. This limitation is not explicitly addressed in their work, potentially affecting the interpretation of their classification performance.

CoLUA, presented by Wei Wen et al. [23], is one of the earliest attempts to automatically identify configuration-related bug reports and extract configuration options. The authors combined NLP techniques with classic ML algorithms, including Naive Bayes, Logistic Regression, and Decision Trees. Despite the promising F1-scores (>0.60 for most models), their dataset is not public and relatively small (900 reports), and their models were evaluated in a closed setup.

Gegick et al. [10] focused on identifying security-related bugs using SAS Text Miner. Although not concerned with configuration, their approach provides valuable insight into early applications of text mining to software defect categorization. However, their work lacks detailed evaluation metrics and reproducibility elements such as dataset accessibility.

Ahmed et al. [1] introduced CaPBug, a framework for automatic bug classification and prioritization. They experimented with various traditional machine learning models using 2,138 bug reports. While their focus was not configuration-specific, their results are strong (F1 score of 0.86), highlighting the effectiveness of well-engineered feature sets even in traditional pipelines. However, the dataset used in their study is not publicly available, which hinders reproducibility and makes it difficult to compare results or replicate the experimental setup in future research.

Xia et al. [25] proposed a framework specifically aimed at predicting configuration bugs through text mining and supervised learning. Their results, however, show an imbalance between the classes, with an F1-score of 0.45 for the configuration class and 0.89 for the non-configuration class. This indicates the difficulty of this task and the limitations of classical approaches.

Xu et al. [26] presented EFSPredictor, a method based on ensemble feature selection tailored to configuration bug detection. Their study used a dataset of over 3,000 reports and relied on multinomial Naive Bayes models. Although focused on configuration and methodologically sound, their reported F1-score remains relatively low (0.57), suggesting room for improvement in model performance.

In contrast to prior work, our approach leverages a general-purpose LLM (Gemini 2.0-flash) for zero-shot classification and further explores fine-tuning a smaller

encoder-based transformer (ModernBERT). To the best of our knowledge, this is the first attempt that combines both LLMs and fine-tuning strategies for the classification of bug reports—specifically targeting configuration-related bugs.

3.3 CHALLENGES AND GAPS ON THE RESEARCH TOPIC

Challenges

To summarize, several challenges arise when conducting research on configuration-related bug classification. These challenges span both conceptual and practical aspects, limiting the reproducibility and scalability of previous approaches. The most critical challenges identified are:

- **Complex definition of configuration:** The concept of software configuration is broad, heterogeneous, and context-dependent. It spans multiple dimensions, including stakeholders, binding times, and artifact types [19], making it difficult to formalize for automated classification tasks.
- **Lack of a clear definition of configuration bugs:** There is no consensus in the literature on what constitutes a configuration bug. This lack of standardization complicates dataset labelling and results in inconsistencies that negatively affect training and evaluation.
- **Scarcity of labelled datasets:** Public datasets that explicitly label configuration-related bugs are rare. Many available datasets either omit the configuration label altogether or suffer from class overlap and ambiguous labelling criteria, limiting their utility for supervised learning.
- **Overlap in datasets due to vague definitions:** The weak and inconsistent definition of configuration bugs in prior work often results in label overlap or misclassification in existing datasets. This reduces their reliability and effectiveness for training models or benchmarking new approaches.

Gaps in the Literature

While bug report classification has been widely studied, existing research rarely focuses on configuration-related bugs as a primary target. Moreover, recent advances in natural language processing, such as large language models, have not been fully explored in this context. Based on our review, we identify several specific gaps that limit the effectiveness and generalizability of current approaches:

- **Limited focus on configuration bugs:** Works such as Catolino et al. [5] include configuration as one among many classes but do not focus their analy-

sis on it. Their dataset shows class imbalance and label overlap, reducing its usefulness for configuration-focused studies.

- **Lack of dataset availability:** Many influential studies, including Wei Wen et al.'s CoLUA [23] and Ahmed et al.'s CaPBug [1], use closed or proprietary datasets. This hinders reproducibility and the ability to benchmark or extend their findings.
- **Under use of recent NLP advances:** Prior research often relies on traditional machine learning techniques (e.g., Naive Bayes, SVMs) and basic text representations (e.g., TF-IDF, bag-of-words). Approaches such as those proposed by Xia et al. [25] and Xu et al. [26] demonstrate modest F1-scores (e.g., 0.45 and 0.57), indicating room for improvement with more advanced models.
- **Lack of generalizability across domains:** Existing methods are often tested on limited or homogeneous datasets, reducing their applicability to larger, real-world contexts.
- **No prior combination of LLMs and fine-tuning:** To the best of our knowledge, no previous study combines large language models in a zero-shot setting with supervised fine-tuning of transformer-based encoders for this specific task. Our work is the first to propose and evaluate this hybrid approach and enabling the first large-scale estimation of configuration bug prevalence in industrial repositories.

3.4 OBJECTIVES

The aim of this master's thesis is not only to produce a practical solution for the classification of configuration-related bugs, but also to contribute meaningfully to the academic and scientific understanding of the problem. Given the growing importance of highly configurable systems and the challenges associated with managing variability-related defects, this work is grounded in both methodological rigour and scientific openness.

The following objectives guide the research conducted in this thesis:

1. **Advance the theoretical understanding of configuration-related bugs in software engineering.** This work aims to contribute to the conceptual and empirical foundation for identifying and classifying configuration-related defects, particularly in highly configurable environments such as Software Product Lines (SPLs), where such bugs are under-explored and difficult to detect.
2. **Investigate the applicability of modern NLP paradigms to software engineering tasks.** By applying Large Language Models (LLMs) and fine-tuned

transformer-based encoders to the classification of bug reports, the thesis explores the potential and limitations of state-of-the-art natural language processing techniques in the context of software artifact analysis.

3. **Compare methodological approaches for automatic text classification.** A rigorous comparative study is conducted between zero-shot classification (using general-purpose LLMs) and supervised fine-tuning (using encoder-based models). This contributes to the methodological discourse in empirical software engineering and NLP.

4. **Promote the construction of high-quality, reusable datasets for configuration bug classification.** Given the lack of publicly available, well-labelled datasets in this domain, this thesis contributes curated and annotated datasets to enable reproducibility, benchmarking, and future research.

5. **Foster open science and reproducibility in empirical software engineering.** This work is driven by a personal commitment to the principles of free, reusable and open science. All developed artifacts—including datasets, source code, prompts, and trained models—are openly shared under permissive licenses to ensure transparency, enable reproducibility, and support the collaborative advancement of knowledge. By contributing openly accessible resources, this work aspires to empower other researchers, practitioners, and the broader software engineering community to build upon and extend these results.

6. **Quantify and contextualize the prevalence of configuration bugs in large-scale software repositories.** The thesis provides an empirical estimation of configuration-related bug frequency in industrial datasets, offering insight into the real-world relevance of this bug category and establishing a baseline for future quality assurance efforts.

3.5 INITIAL APPROACH AND DATASET LIMITATIONS

Following the identification of key gaps in the existing literature particularly around reproducibility, the lack of modern natural language processing (NLP) techniques applied to the classification of configuration bugs, and the absence of high-quality datasets, we formulated an initial research direction focused on leveraging large language models(LLMs) to classify configuration-related bug reports.

While natural language processing (NLP) has been used in prior studies to classify bug reports[1, 5, 23, 25] (including some efforts that target configuration-related bugs) these approaches often faced challenges in terms of reproducibility, limited access to labelled datasets, or constraints in model generalization. Many relied on traditional machine learning methods, which were appropriate for their

time but are now complemented by more advanced techniques such as large language models (LLMs), offering new capabilities in understanding and classifying unstructured natural language data.

A key design decision in our work was to focus on bug reports rather than bug-fixing commits as the unit of classification. This choice was motivated by our review of prior research, where we noted that some influential studies in the field used bug-fixing commits to classify bug types[**Abal2017**]. However, bug-fixing commits typically appear at later stages of the development process and are not available during the early phases of bug triaging. In contrast, bug reports are available earlier, tend to be written in natural language, and often contain rich contextual information that can reveal configuration-related aspects that may not be directly observable in code changes. For this reason, we consider bug reports to be a more valuable input for classification in the context of our work.

We aimed to classify these bug reports using the taxonomy proposed by Siegmund et al. in their work *Dimensions of Software Configuration*[19]. This taxonomy articulates a rich and descriptive conceptualization of configuration contexts, including aspects such as feature selection, environment setup, runtime parameters, and tool-specific settings. Using this taxonomy as a foundation promised to provide a structured approach for classification while also connecting our findings to ongoing work in the configuration research community.

However, translating this taxonomy into a practical labelling task surfaced significant challenges. First, the absence of a clear and operational definition of what constitutes a configuration bug in existing literature made the labelling process inherently subjective. Secondly, we observed that even in datasets where configuration was nominally included as a class (e.g., Catolino et al.[5]), the boundaries between configuration and other types of bugs were often ambiguous or overlapping. This ambiguity undermines the reliability of these datasets for training or benchmarking models.

To address these issues, we first had to find a reliable dataset that had already classified configuration bug reports, but this was not an easy task since dataset didn't consider this label or had overlapping issues. So, instead of classifying configuration bug reports into the described taxonomy, first we needed to create or curate a dataset of configuration bug reports.

In summary, this initial approach set the stage for our broader contribution: combining rigorous data curation, reproducibility-focused experimentation, and the application of state-of-the-art LLMs to enable reliable classification of configuration bugs using natural language analysis.

Table 1: Comparison of related works on configuration bug classification

Research	**CS**	**PD**	**# Bugs**	**Techniques**	**Tools / Models**	**LLM**	**ModernBERT**	**F1**
G. Catolino et al.[5]	✗[1]	✓	1,280	ML	SVM, RF, NB, LR[2]	✗	✗	~0.64
W. Wen et al.[23]	✓	✗	900	NPL + ML	NB,LR,DT[3]	✗	✗	>0.60[4]
M. Gegick et al. [10]	✗	✗	N/A	Text mining	SAS Text mining	✗	✗	N/A
H. A. Ahmed et al. [1]	✗[1]	✗	2,138	NLP + ML	DT, RF, NB, LR	✗	✗	0.86
X. Xia et al. [25]	✓	✗	N/A	Text mining + ML	NB multinomial, NB, kNN, Bayesian network, SVM	✗	✗	0.45[5]
B. Xu et al. [26]	✓	✗	3,203	NPL + Feature selection	NB mulinomial	✗	✗	0.57
This Work	✓	✓	1,331 + 300k	LLM + fine-tune	Gemini, ModernBERT	✓	✓	0.81

CS: Is the research about specifically about configuration related bug?
PD: Is the dataset publicly available?

[1] Consideres Configuration in its taxonomy of classification.

[2] In order: Support Vector Machines(SVM), Random Forest(RF), Naive Bayes(NB) and Logistic Regression(LR).

[3] Decision Tree(DT).

[4] F1-score greater than 0.6 in 16 out of 18 models, but does not provide an average F1-score.

[5] F1-score for the class Configuration. The F1-score for the no-configuration class was 0,89.

4 CONFIGURATION BUG CLASSIFICATION

In this section, we discuss the approach used to obtain a labelled dataset, the different approaches that we have followed in the experimentation phase, and how we have used the different models to evaluate the percentage of configuration-related bugs in a dataset.

This research followed an iterative development and evaluation approach, where each phase built upon the insights and limitations of the previous one to incrementally refine the tools, models, and methodology. Our objective was to design an effective pipeline for classifying configuration-related bug reports using large language models (LLMs), starting with the construction of a querying application, followed by prompt optimization, and culminating in the fine-tuning of transformer-based models on curated datasets.

4.1 PHASE 1: APP DEVELOPMENT FOR MODEL QUERYING

We began by designing and implementing a flexible application capable of querying various LLMs with bug report data. The application was first tested in a local environment using a small-scale LLM to validate the architecture and query structure. This initial prototype included a JSON configuration file that allowed researchers to specify critical parameters such as the LLM to be used (e.g., Gemini, LLaMA, OpenAI-compatible models), the file path to the dataset of bug reports, the temperature and token limits, and, importantly, the prompt template.

To have a better control of the response and input offered to the LLM, we decided to process the dataset and choose only the valuable information and not the entire report. Using the title and description, we prompt the model to classify a bug report. This prompt was also refined as we will explain in the following section. As a result, we declared that the response should be a JSON object with a class and a reasoning, explaining why this bug report should be classified as the determined class. This gave us a better understanding for the following iteration on what where the weaknesses or failures of our prompt.

To mitigate inconsistencies in LLM outputs, where models might return noisy or verbose responses, the application implements a specialized evaluation strategy based on edit distance. When the model output deviates from the expected class labels (e.g., "Configuration" or "Other"), the tool automatically resolves the predicted class to the nearest valid label using Levenshtein distance. This mecha-

nism addresses the common issue of non-canonical string predictions by aligning outputs to a fixed label set prior to computing performance metrics.

In addition, the application was designed with extensibility in mind, supporting any LLM that conforms to the OpenAI API specification. This abstraction enabled the seamless integration of different LLM providers and models while maintaining a consistent interface for querying and data collection.

The evaluation process computes standard classification metrics including precision, recall, and F1-score. To complement these, we introduced an additional metric: the error in class proportion. This measures the absolute difference between the true and predicted prevalence of configuration-related bugs in the dataset, offering insights into the model's calibration—particularly relevant in settings with class imbalance.

Finally, to avoid redundant computation and ensure traceability, experiment results and metrics are stored in structured directories based on a hash of the configuration. When rerunning an existing experiment, the system automatically loads the previous results, optimizing computational efficiency while preserving the reproducibility of results.

4.2 PHASE 2: PROMPT ENGINEERING AND ZERO-SHOT CLASSIFICATION

4.2.1 *First iteration: Initial dataset and experiments*

After identifying key gaps in the state of the art, particularly concerning datasets, we observed a notable scarcity of publicly available and well-labelled datasets focused on configuration-related bug reports. This limitation posed a challenge for the development and evaluation of classification models within this domain. To address this, we broadened the scope of our search to include datasets that, while not originally created for this specific purpose, included tags or metadata referring to configuration-related bugs.

This extended exploration led us to the *NABATS dataset*, a dataset with 1280 bug reports of 119 popular projects belonging to three ecosystems such as Mozilla, Apache, and Eclipse where bugs were classified into nine classes, one of them being Configuration. Although we initially identified inconsistencies and overlapping class definitions in the original version, the dataset served as a valuable starting point for the first iteration of our experiments and prompt design. As such, we adopted NABATS for the initial phase of our work to guide prompt refinement and early model evaluation.

To use this dataset in our experimentation, we transformed the labels on it, renaming all non-configuration-related bugs with the label *Other*, and leaving the Configuration label on the rest(represented on the first step on Figure 4). As a result we got a dataset with 1094 bug reports not related to configuration and

186 bug reports related to configuration. Additionally, since some bug report summaries provided limited contextual information, we decided to enrich the dataset by incorporating the full description field.

4.2.2 *First prompt iteration*

Once the infrastructure was established, and that a first version of a dataset was produced, we iteratively developed and refined the prompts used for zero-shot classification. The initial prompt was the following:

> *The user will provide information about a bug report. This information includes: Bug-ID, Project, Summary, Description, Link, and Environment. Classify the bug report into* ***Configuration Bug Report*** *or* ***Other****, where the first class indicates that the bug report is related to a configuration bug or issue, and the second class indicates that the bug report is* ***not related*** *to configuration issues, but instead concerns other matters such as database errors, feature requests, functional bugs, GUI-related issues, network problems, performance, or security flaws. If the bug report refers to any of those themes, classify it as* ***Other****.*
>
> *The first category (****Configuration Bug Report****) refers to bugs concerned with building configuration files. Most of them involve (i) external libraries that need to be updated or fixed, or (ii) incorrect directory or file paths in XML or manifest artifacts.*
>
> *Understand that bug reports can be either* ***Configuration Bug Report*** *or* ***Other****.*
>
> ***Reply ONLY with one of the two classes****, using no more than three words: "Configuration Bug Report" or "Other".*

In this prompt, we described the information given to the model, the classification task, and clear definitions for both configuration-related and non-configuration-related bug reports, obtained from Catolino's et al [5] definition of Configuration bugs. However, this initial setup revealed important limitations (as shown in Table 2). Although the definition of configuration was broad and well-structured, it lacked sufficient detail to help the model detect more subtle or ambiguous cases. As a result, the model tended to classify most bug reports as belonging to the "Other" class. This led to a high precision of 0.923 for that class, meaning the model was often correct when predicting non-configuration bugs—but this result was also influenced by the imbalance in the dataset. Despite this, the recall for the "Other" class was relatively low (0.375), indicating that many true non-configuration reports were not identified.

On the other hand, for the configuration-related class, the model achieved a high recall (0.817), meaning it successfully identified most true positives. However, its precision was very low (0.184), showing that many reports it flagged as configuration-related were actually not, which severely impacted the F1-score for that class (which resulted in 0.300). Overall, the model reached an accuracy of

only 0.440, reflecting its limited ability to make balanced predictions. These results suggest that the initial prompt did not offer enough clarity or discriminative information to help the model reliably distinguish between the two classes, especially in complex or borderline cases.

In addition to the prompt's conceptual limitations, the examples embedded within it sometimes introduced unintended biases, guiding the model toward specific keywords or patterns that skewed its decisions. Furthermore, the definitions of configuration and non-configuration reports explicitly referenced elements such as configuration files and external libraries. This inadvertently led the model to overemphasize these terms when making classifications, often flagging unrelated reports as configuration-related simply due to lexical matches. These observations highlight the critical importance of precision, balance, and neutrality in prompt design, prompting us to iteratively refine the prompt structure and content in later experiments to improve classification accuracy and generalizability.

Table 2: Classification performance metrics - First iteration: Simple configuration definition using NABATS dataset

Class	**Precision**	**Recall**	**F1-score**	**Support**
Configuration Bug Report	0.184	0.817	0.300	186
Other	0.923	0.375	0.533	1081
Accuracy		0.440		
Macro avg	0.553	0.596	0.416	1267
Weighted avg	0.814	0.440	0.499	1267

At the same time, we analysed the justifications provided by the models when selecting a particular class, which offered valuable insight into their decision making processes. This qualitative analysis revealed a critical issue with the dataset used in this iteration. We observed that several bug reports classified under non-configuration categories still contained configuration-related elements and vice versa.

Upon reviewing specific misclassifications, we found multiple cases where the labelling in the original dataset appeared inconsistent.

For instance, in the first bug report (Figure 3a), the reporter describes the necessity to increase the maximum buffer size, indicating an issue with a configuration attribute, but in the original dataset, this was classified as "Add issue"(a label defined in the original research [5]) which might correct but could also be considered a configuration-related bug.

Need to increase the maximum buffer size for HTTP header from 16k to 64k.
Apache - Tomcat Connectors - Bug report: **42003**
Description: Currently the maximum buffer size for HTTP headers in the native JK connector is 16k. However, this size is not sufficient in case the HTTP header is larger than 16k (i.e. large Kerberos Tickets). If such a case arises an error of the type "[error] jk_isapi_plugin.c (1135): failed to init service for request" gets raised. For this reason the maximum buffer size, which currently is hard- coded to 16k, must be increased to 64k.

(a) Misclassified as "Add issue" in the original dataset

Setting accessibility in Java
Apache Jira — Apache FOP - Bug report **49808**
Description: When setting accessibility with :
A. userAgent.setAccessibility(true)
and then generating PDF, an exception occurs :
- - - - -
ERROR org.apache.fop.area.AreaTreeModel - Error while rendering page 1 java.lang.NullPointerException at org.apache.fop.render.pdf-
.PDFPainter.prepareImageMCID(PDFPainter.java:152)
- - - - -
It seems that logicalStructureHandler of PDFPainter is null.
The cause is that the logicalStructureHandler is set when calling :
B. FopFactory.newFop(String outputFormat, FOUserAgent userAgent, Output-Stream stream)
which calls :
PDFDocumentHandler.startDocument() where it tests accessibility of the attached FOUserAgent.
Issue is due to the fact that at the moment calling "B", we couldn't have correctly set "A" so, the logicalStructureHandler of PDFPainter remains null.
A solution is to change method FopFactory.setAccessibility() to public, so that we can set accessibility before calling "B".

(b) Misclassified as "Test Code-related issue" in the original dataset

Figure 3: Examples of misclassified bug reports that could be interpreted as configuration-related.

In the second case presented (Figure 3b), the summary does not give a detailed enough representation of the problem, but the description indicates that with a valid configuration of the system i.e. setting the accessibility with *A. userAgent.setAccessibility(true)* and trying to generate a PDF, the execution resulted on a null pointed exception. In the original dataset this was classified as "Test Code-related issue", but as we can deduce, this is a problem related to the configuration of the userAgent.

This overlap introduced ambiguity in class boundaries, as configuration-related aspects were often embedded in reports labelled as performance, functionality, or even documentation issues. As a result, the model's misclassifications were not only due to prompt limitations but also stemmed from inconsistencies and noise in the training data. This finding underscored the need for a more precise and mutually exclusive labelling scheme, prompting the creation of a curated and resampled version of the dataset with clearly disjoint classes.

4.2.3 *Second iteration: Dataset curation and extension*

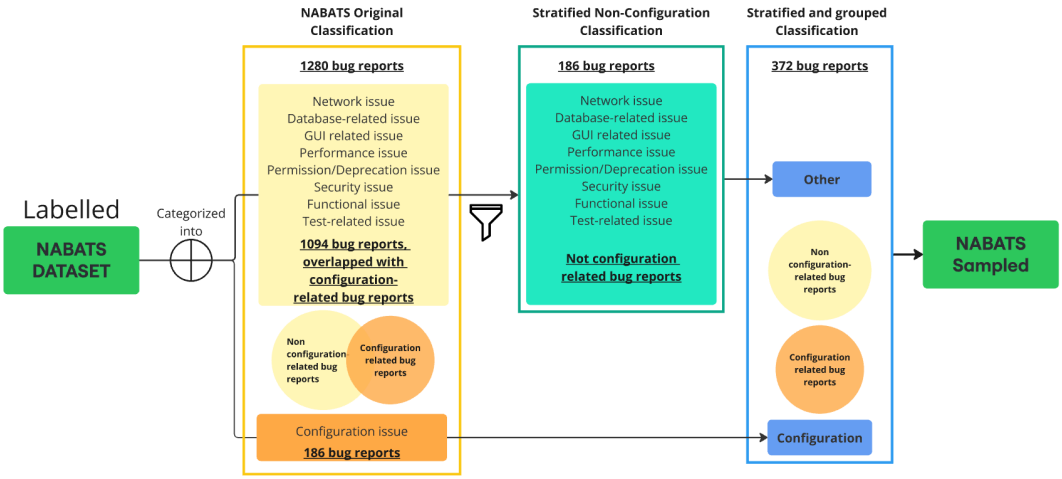

Figure 4: Data curation of Catolino et al.'s dataset (referred as NABATS)[5] to a revised sampled version.

To address the issues identified in the NABATS dataset, we decided to carry out two parallel tasks.

First, we curated the NABATS dataset by manually reviewing its bug report labels. The goal was to eliminate instances with ambiguous or overlapping classifications and retain only those reports for which we could confidently determine whether they were related to configuration or not. This process (represented on

Figure 4) allowed us to construct a cleaner subset of NABATS, suitable for more reliable experimentation, with a total of 372 bug reports (186 for each class).

Second, we built a new dataset from a software repository that we knew in detail, allowing us to confidently assess whether each bug report was configuration-related or not. By leveraging our domain knowledge of this specific project, we ensured high labelling accuracy and consistency.

The chosen project is UVLHub [17][1], web-based repository of feature models written in the Universal Variability Language (UVL). It offers a user-friendly front end to support the search, upload, storage, and management of variability model datasets. Additionally, UVLHub integrates with Zenodo, enabling the permanent archiving of datasets in accordance with open science principles. As a Software Product Line (SPL) platform, UVLHub is inherently highly configurable, making it an ideal candidate for studying configuration-related bugs, especially those stemming from feature interactions, model constraints, or deployment variability.

To get information about the bug reports of this project, we used Github's API [2] to request all the bug reports and store them in a file with information about the summary, description, and tags of the said bug report. Once we had the bug reports from UVLHub, we proceeded to manually label them into *Configuration* or *Other*, where "Configuration" indicates that a bug report is considered to be related to a configuration bug and "Other" indicates that the bug is not related to configuration.

The purpose of the UVLHub dataset is threefold. First, it allows us to evaluate the effectiveness of the classification models on a different type of software project, namely a highly configurable web-based platform. Second, it serves as a reliable benchmark due to the high confidence we have in the accuracy of its manual labelling, which was performed by researchers with in-depth knowledge of the system. Finally, it enables a controlled setting to design and refine prompts for zero-shot classification, ensuring that the evaluation is not influenced by noisy or ambiguous labels.

4.2.4 *Second prompt re-design*

To address the concerns raised by the prompt itself, we integrated a more comprehensive definition of configuration inspired by Siegmund et al.[19], encompassing aspects such as feature selection, environment variables, parameter settings, and deployment contexts. The refined prompt explicitly defined configuration bugs and included examples of configuration-related terms and failure modes, which significantly improved the clarity of the classification task for the model. The prompt used in this iteration, was the following:

1 https://github.com/diverso-lab/uvlhub
2 https://github.com/

> *Software configuration is the process of defining, managing and modifying parameters, options and artifacts that affect the behavior, execution and infrastructure of a software system. It encompasses a broad spectrum of activities including customization of functionality, management of runtime environments, and customization of infrastructure and development tools.*
>
> ***What is a Configuration Bug Report?***
>
> *A configuration bug report describes an issue caused by misconfigured settings, incorrect parameter values, missing configuration files, or unintended interactions between different configuration options. These bugs typically manifest as system failures, performance degradation, security vulnerabilities, or unexpected behavior due to incorrect environment settings. The user will provide information about a bug report, including: Bug-ID, Project, Summary, Description, Link and Enviroment. Consider if this bug report is related to configuration or not and reply with a JSON with the following structure : "class": "Configuration" | "Other", "reason": "text", where class can be either 'Configuration' or 'Other' and "reason" is a brief explanation of no more than 100 words about why you classified the bug report into that class. Please, reply with a JSON object as explained and avoid line breakers.*

This prompt was tested on both the curated NABATS dataset (from now on NABATS-sampled) and the UVLHub dataset to evaluate results on two different datasets.

Although this prompt included a general definition of configuration, the way it introduced configuration-related bugs was overly simplistic and lacked sufficient specificity. The definition failed to capture the key characteristics and indicators that typically distinguish configuration bugs from other types of bugs. As a result, the model's performance under this prompt was limited, yielding suboptimal results. Nevertheless, it still outperformed earlier baselines (with a macro F1-score of 0,535), indicating that even a basic framing of the task can be beneficial when combined with a structured prompt.

4.2.5 *Third iteration: Grouped dataset*

As part of the ongoing prompt refinement process, we decided to provide a more robust and precise definition of software configuration. In parallel, we aimed to strengthen the LLMs understanding of the task by explicitly outlining the types of characteristics and topics that could indicate a configuration-related bug. This approach was intended to guide the model more effectively in distinguishing between configuration and non-configuration bugs.

The resulting prompt was as follows:

> ***Definition of Configuration*** *Configuration in software systems refers to the process of setting parameters, options, or environment variables that influence the behaviour, performance, or functionality of a system. It can be classified into different types, including:*

- ***Domain-specific configuration*** *– tailors functionality for end-users, often involving feature toggles.*
- ***Technical configuration*** *– involves infrastructure, deployment, or system settings such as environment variables, databases, or cloud infrastructure.*
- ***Development configuration*** *– configures development tools, CI / CD pipelines, and build processes.*

What is a Configuration Bug Report?

A configuration bug report describes an issue caused by misconfigured settings, incorrect parameter values, missing configuration files, or unintended interactions between different configuration options. These bugs typically manifest as system failures, performance degradation, security vulnerabilities, or unexpected behaviour due to incorrect environment settings.

Task

Given a bug report, your task is to determine whether it qualifies as a ***configuration bug report****. A bug report is likely a configuration issue if:*

1. ***It mentions configuration artifacts*** *– such as config files, environment variables, registry settings, database settings, or deployment scripts.*
2. ***It describes incorrect or missing configurations*** *– for example, missing or incorrect values in configuration files leading to failures.*
3. ***It relates to system setup, deployment, or runtime settings*** *– such as issues caused by cloud, server, or infrastructure configurations.*
4. ***It involves unintended interactions between configuration settings*** *– where multiple configuration parameters lead to unexpected system behaviour.*
5. ***It does not primarily involve code defects*** *– meaning the problem is due to how the system is configured rather than a flaw in the source code itself.*

Response Format *Provide the classification in* ***JSON format*** *as follows:*

{ "class": "Configuration" | "Other", "reason": "<brief explanation for the classification>" }

As a first approach to applying models to large datasets, we also offer *the Eclipse Dataset*[3], a comprehensive collection of bug reports extracted from Eclipse's Bugzilla. This dataset consists of 301,465 bug reports covering key Eclipse projects such as Platform, JDT, CDT, PDE, Equinox, BIRT, Mylyn, TPTP, and Papyrus. Each bug report includes all the available information about them, such as Bug ID, creation time, status, resolution, severity, summary, description, comments, change history, and attached files among others.

The Eclipse Dataset was generated using a custom Command Line Interface (CLI) tool specifically designed for extracting comprehensive information from Bugzilla repositories. This tool, called Issuex[4], automates the process of querying Bugzilla's

3 https://doi.org/10.5281/zenodo.15348468
4 https://github.com/diverso-lab/Issuex

APIs, ensuring a consistent and thorough collection of data across different instances of Bugzilla.

Issuex has two commands that can be used to obtain the bug reports from a repository:

- ***issuex run***. This command uses the information specified in the configuration file. The user must specify status and issue resolution as input parameters. The tool obtains a list of requests and processes it to obtain detailed information.
- ***issuex run:default***. This command is identical to the previous one. The difference is that the user does not need to specify the input parameters, default ones are used.

By using this tool, we produce *The Eclipse Dataset*, that represents one of the largest publicly available collections of bug reports specifically focused on the Eclipse ecosystem, which is itself a major open-source software platform. Its scale, diversity, and level of detail make it a valuable resource for evaluating automated classification models, analysing defect patterns within a highly configurable system, and supporting empirical studies in Mining Software Repositories (MSR). The dataset's structured format and rich metadata enable reproducibility and facilitate its integration into a variety of software engineering research workflows.

As the key fields for the classification task in our models are the summary and description, we excluded unnecessary information from the dataset to enhance its clarity and usability for the models. As a result, we get a large dataset from various projects of Eclipse with details about the summary and description of each one of the 301,465 bug reports.

4.3 PHASE 3: SUPERVISED CLASSIFICATION

In this phase, we explored a supervised learning approach to classify configuration-related bug reports. With this new methodology, we aimed to assess whether fine-tuning a model on domain-specific and well-labelled data could result in more consistent and reliable performance.

To support the supervised classification phase, we selected ModernBERT as the base encoder model due to its efficiency, adaptability, and ease of integration in constrained computing environments. ModernBERT provides a balanced architecture that enables effective learning from domain-specific datasets while maintaining a relatively low computational footprint. Its design facilitates fast fine-tuning and inference, making it particularly suitable for local environments such as personal workstations or academic lab setups. This aligns with our objective of developing a reproducible and resource-efficient approach that can be adopted with-

out requiring access to large-scale infrastructure. Furthermore, ModernBERT integrates seamlessly with standard transformer toolkits, allowing for streamlined experimentation and deployment within our classification pipeline.

Following the experiments presented in the zero-shot classification, the classification task was framed as a binary problem, distinguishing between two categories: Configuration and Other.

The process of classifying a bug report into one of these categories starts by preparing the input data. Bug reports were sourced from CSV files, and any entries containing uncertain or ambiguous labels were removed to ensure the quality of the training set. Each report was represented by concatenating its title and description into a single structured string in the format: "SUMMARY: [title] \n \n DESCRIPTION: [description]". This unified input was designed to preserve the typical structure of bug reports and provide the model with both brief and contextual information in a consistent way.

The data was then split into training and testing subsets using an 80/20 stratified split, ensuring that the class distribution was preserved across both sets. Then, tokenisation was performed using the ModernBERT tokenizer, with inputs truncated or padded to a fixed maximum length. Class labels were mapped to integer values to create a numerical target for the classification task.

A lightweight dataset class was implemented using PyTorch to store the tokenised inputs and associated labels. For model training and evaluation, we used the HuggingFace Trainer API[5], which offers a flexible interface for fine-tuning transformer models. Training was carried out using the AdamW optimiser, with a linear learning rate schedule and regular weight decay to prevent over-fitting. The hyperparameters, such as learning rate, batch size, and number of epochs, were selected to fit within the constraints of a local environment, without requiring access to high-performance computing resources.

In addition to the standard metrics such as accuracy and F1-score and following the steps defined on the zero-shot classification, we defined a custom evaluation function to track the macro F1-score as well as the per-class performance. We also monitored the proportion of predictions classified as Configuration and computed the absolute difference from the true distribution in the test set. This allowed us to assess how well the model preserved the overall class balance in addition to its classification accuracy.

Throughout training, evaluation was conducted at the end of each epoch, and the best-performing model based on validation results was retained. The final metrics were aggregated into a structured format to enable comparison with other experimental setups. This supervised classification pipeline proved effective and reproducible, and was successfully executed in a resource-constrained local setting.

5 https://huggingface.co/docs/transformers/main_classes/trainer

The results showed a clear improvement over the zero-shot models, particularly on datasets with high-quality annotations and well-separated classes (as represented on Table 4). For instance, the fine-tuned model achieved a macro F1-score of 0.812 on the combined dataset, surpassing the performance of the best zero-shot configuration. This suggests that fine-tuning on curated data enables the model to better capture the specific patterns and indicators associated with configuration-related bugs.

In addition, the supervised model demonstrated greater stability across datasets and was not sensitive to prompt phrasing, since it relied on a fixed input structure learned during training. This made it more suitable for practical use cases where classification consistency is critical. These outcomes highlight the value of labelled data and task-specific model adaptation when applying NLP techniques to software engineering problems.

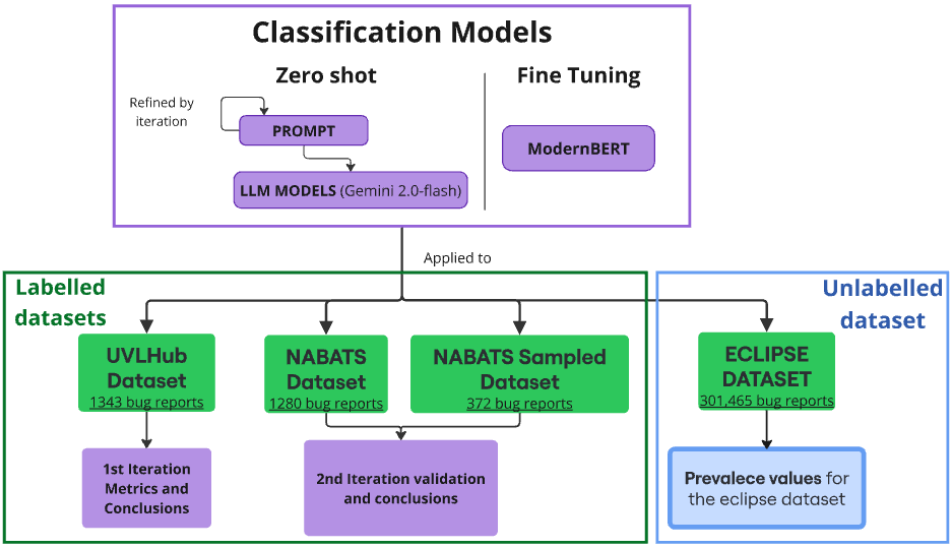

Figure 5: Representation of the different experiments realised

As a result, we developed an experimental pipeline (illustrated in Figure 5) comprising two distinct classification approaches: a zero-shot model and a supervised model. Both were applied to three different datasets in order to evaluate their performance under varying conditions. During the process, one dataset had to be discarded from the final evaluation due to overlapping class definitions, which introduced label inconsistencies and compromised the reliability of the results.

Upon analysing the results across the remaining datasets, it became evident that the supervised model consistently outperformed the zero-shot approach in terms of precision, recall, and F1-score. Its ability to learn from labelled examples allowed it to capture domain-specific patterns more effectively, resulting in higher classification accuracy and more reliable generalisation. These findings reinforce

the practical advantages of supervised fine-tuning, particularly when high-quality annotated data is available and consistency across diverse data sources is essential.

Having developed a model capable of accurately predicting whether a bug report is configuration-related with strong F1-scores across different datasets, we proceeded to evaluate its applicability at scale. To this end, we applied the fine-tuned supervised model to a large dataset comprising over 300,000 bug reports extracted from Eclipse projects. This dataset originates from a previous research effort but was further refined and adapted for the current classification task. In particular, it was preprocessed to retain only relevant fields such as summary and description, ensuring compatibility with the input structure expected by the model. This large-scale evaluation allowed us to assess the generalisability of our classifier in a real-world, industrial context and to estimate the prevalence of configuration-related bugs within a highly configurable ecosystem like Eclipse.

When applying the supervised model to the large Eclipse dataset, the classifier processed each bug report based on its summary and description, assigning a label of either Configuration or Other. The outcome of this large-scale evaluation revealed that approximately 36% of the bug reports were predicted as configuration-related. This proportion is significant and aligns with previous empirical studies[27] that have highlighted the high prevalence of configuration errors in large and complex systems. The result not only confirms the relevance of configuration-related bugs in industrial software repositories but also demonstrates the practical utility of the trained model in analysing large volumes of unlabelled data. This large-scale estimation provides new insights into the distribution of configuration bugs and supports future efforts in prioritising their detection, triaging, and resolution in configurable systems.

Table 3: Classification performance metrics – Refined prompt results across three datasets

Dataset	Class	Precision	Recall	F1-score	Support
UVLHub	Configuration Bug Report	0.430	0.799	0.559	164
	Other	0.968	0.851	0.906	1167
	Accuracy		0.844		
	Macro avg	0.699	0.825	0.732	1331
	Weighted avg	0.902	0.844	0.863	1331
NABATS-sampled	Configuration Bug Report	0.674	0.699	0.686	186
	Other	0.687	0.661	0.674	186
	Accuracy		0.680		
	Macro avg	0.680	0.680	0.680	372
	Weighted avg	0.680	0.680	0.680	372
Combined dataset (UVLHub + NABATS-sampled)	Configuration Bug Report	0.524	0.746	0.616	350
	Other	0.926	0.825	0.873	1353
	Accuracy		0.809		
	Macro avg	0.725	0.785	0.744	1703
	Weighted avg	0.844	0.809	0.820	1703

Table 4: Performance comparison between Gemini 2.0-flash as zero-shot classifier and ModernBERT-base as a supervised classifier.

Dataset	**Conf. Ratio**	**Model (G/M)**	**Macro F1**	**F1 Conf.**	**F1 Other**
UVLHub	0.124	G	0.732	0.559	0.906
		M	**0.823**	**0.694**	**0.952**
NABATS	0.147	G	0.515	0.332	0.698
		M	**0.644**	**0.395**	**0.894**
NABATS-sampled	0.507	G	0.680	0.686	0.674
		M	**0.732**	**0.750**	**0.714**
NABATS + UVLHub	0.206	G	0.744	0.616	0.873
		M	**0.812**	**0.697**	**0.927**

Conf. Ratio indicates the ratio of configuration-related bugs present on the dataset.
Model Indicates the model being used in the experiment G (gemini-2.0-flash) or M (ModernBERT-base)

Part IV

FINAL REMARKS

5 CONCLUSION AND FUTURE WORK

In this chapter, we summarize the main findings of our study on the classification of configuration-related bug reports using large language models (LLMs) and fine-tuned encoder-based transformers. We reflect on the effectiveness of both zero-shot and supervised approaches across multiple curated datasets and real-world repositories. Furthermore, we discuss the implications of our results for software quality assurance and highlight avenues for future research aimed at improving bug classification accuracy and scalability.

5.1 CONCLUSIONS

In this work, we demonstrated the effectiveness of leveraging a general-purpose large language model (LLM) as a zero-shot classifier for distinguishing between configuration-related and non-configuration bug reports. By applying Gemini 2.0-flash to this task, we provided a novel perspective on the potential of LLMs in software bug classification, showcasing their ability to perform reasonably well without task-specific fine-tuning.

Additionally, our study presents the use of ModernBERT, a smaller encoder-based transformer model fine-tuned specifically for the task of classifying configuration-related bugs. While not as large as general-purpose LLMs, ModernBERT demonstrated strong performance across both curated datasets, achieving a balanced trade-off between accuracy and efficiency. Its success highlights the benefits of task-specific adaptation, suggesting that fine-tuning smaller models on well-labelled domain data can yield reliable results without the need for extensive computational resource.

During the evaluation of the models, we also observed an important consideration: larger descriptions, particularly those containing logs or detailed system information, tended to reduce the performance of the model considerably. This suggests that while more detailed descriptions may seem to offer richer context, they can introduce noise or excessive complexity that complicates the automatic classification task. This finding highlights the need to optimize the structure and content of bug reports, focusing on concise and relevant information to improve model performance.

Our contributions are further solidified by the introduction of several new datasets, which have not been available before. These datasets, enriched through careful curation and stratified sampling, not only allowed for a more rigorous evaluation of

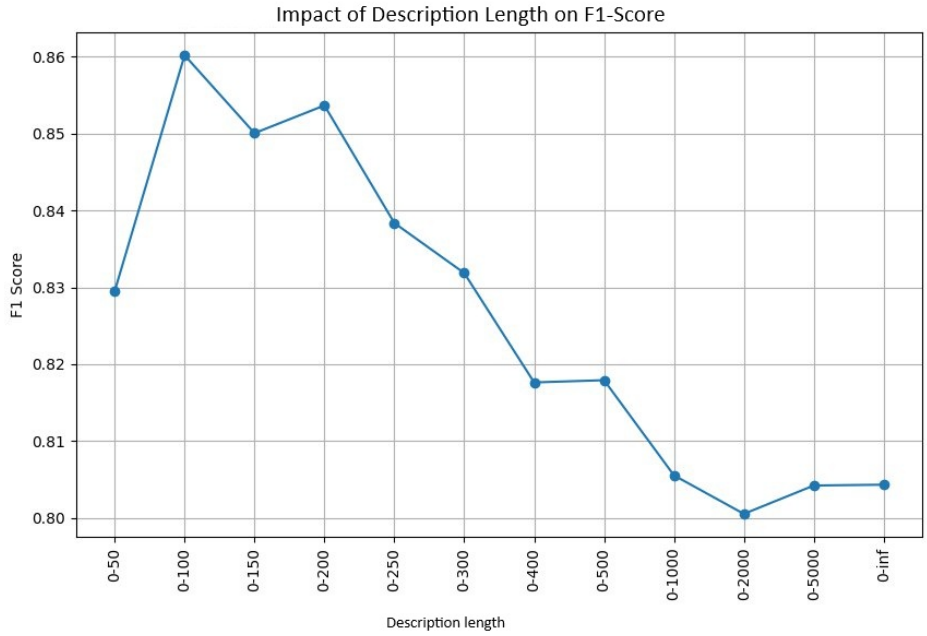

Figure 6: Evolution of F1-score when including bug reports with longer descriptions.

classification methods but also provided a valuable resource for future research in bug report analysis.

Moreover, we offer the first estimation—based on a substantial dataset—of the importance of configuration-related bugs. This finding opens new avenues for research, as understanding the prevalence and impact of configuration bugs can drive improvements in software quality assurance and maintenance processes.

Finally, all artifacts associated with this study, including datasets, code, and experimental pipelines, have been made publicly available. This comprehensive experimentation stack is intended to facilitate reproducibility and foster further exploration in the field, ultimately contributing to a more robust understanding of bug classification challenges and solutions.

5.2 LIMITATIONS AND EXTENSIONS

This study makes a significant step forward in the classification of configuration-related bug reports using large language models and fine-tuned encoders. Nevertheless, several limitations must be considered when interpreting the findings.

One central limitation stems from the ambiguity in the definition of configuration. As highlighted by Siegmund et al. [19], configuration is a pervasive but ill-defined concept in software engineering. It may refer to a wide range of artifacts and decisions, from build-time feature selection to runtime parameters and

environmental settings. This conceptual vagueness introduces subjectivity during dataset labelling and complicates the design of models that must generalize over such a broad space. Even human annotators may disagree on whether a bug is configuration-related, especially when it coexists with other bug types such as performance or functional issues. This subjectivity inevitably propagates into the learning process and model outputs.

Another methodological threat relates to the prompt-based nature of zero-shot classification. The performance of general-purpose LLMs such as Gemini 2.0-flash is highly sensitive to prompt phrasing. Minor variations in wording can significantly affect classification outcomes, limiting the reproducibility and stability of results. Moreover, LLMs may exhibit biases toward dominant language patterns in their training data, which do not necessarily align with the characteristics of configuration bugs.

The choice of model architecture also introduces a limitation. While this study focused on Gemini 2.0-flash and ModernBERT, we acknowledge that other LLMs and transformer variants, such as GPT-4, Claude, or DeBERTa, may offer improved performance or different trade-offs. However, an intentional design decision in our work was to prioritize models that are freely accessible and compatible with local execution under limited computational resources. This choice enhances the practical applicability of our approach for researchers and practitioners who may not have access to commercial APIs or large-scale cloud infrastructure. Nonetheless, it limits the scope of comparison and leaves open the question of how more powerful proprietary models might behave in this classification task.

In terms of data, dataset representativeness poses a notable limitation. The UVL-Hub dataset, although manually curated and high quality, is drawn from a single project and may not reflect the variability found in other systems or domains. Similarly, the NABATS Sampled dataset, although more diverse, is derived from pre-existing labels that may embed legacy biases. While we attempted to generalize by applying our models to a large dataset of Eclipse bug reports, this too reflects the practices of a specific ecosystem, with its own conventions and tooling.

The interpretation of performance metrics must also be nuanced. The macro F1 score of 0.812 suggests balanced performance across classes, but it does not capture aspects such as model calibration, confidence, or robustness. Furthermore, our large-scale estimation that approximately 36% of Eclipse bug reports are configuration-related is based on a single model's prediction, without confidence intervals or extensive human validation. While this figure offers insight into the prevalence of configuration bugs in a large real-world project, **it should be treated as an approximation rather than an exact measurement**. Differences in model calibration, dataset composition, or annotation quality could all influence this estimation As such, it should be considered an indicative approximation rather than a definitive measurement.

5.3 FUTURE WORK

Building upon the contributions and limitations of this study, several promising opportunities emerge for future work, both in terms of research exploration and practical deployment.

While our work focused on resource-efficient, open-access models such as Gemini 2.0-flash and ModernBERT, future studies should consider expanding the evaluation to include more recent and diverse architectures—both open-source (e.g., RoBERTa, DeBERTa, DeepSeek) and proprietary (e.g., GPT-4, Claude). A systematic comparison across models would provide deeper insights into the trade-offs between computational efficiency, accuracy, and generalization in the context of configuration bug classification.

To improve the robustness of classification models, future work should focus on collecting and annotating bug reports from a broader variety of software domains, such as proprietary projects, cloud-native architectures, or mobile applications. This would allow the training of more generalizable models and facilitate better cross-domain transferability, reducing the risk of ecosystem-specific bias.

Manual dataset curation, while high quality, is inherently limited in scale. A key future direction is the development of semi-automated annotation pipelines, combining weak supervision, active learning, or bootstrapped LLM classifiers with human validation. This would accelerate dataset creation while maintaining label accuracy, enabling the training of larger and more representative models.

Given the sensitivity of zero-shot models to prompt design, future work could explore prompt-tuning techniques or reinforcement learning-based optimization of prompts. Additionally, dynamically adapting prompts based on the characteristics of the bug report (e.g., type of component, presence of logs) could enhance performance and mitigate brittleness.

The promising results of this study open up a wide range of exciting opportunities for future research and practical innovation. As large language models and fine-tuned transformers continue to evolve, so too does their potential to transform how we understand and manage software quality. With continued exploration across models, domains and methodologies, this line of work can lead to more intelligent, efficient, and accessible tools for identifying configuration bugs at scale. Given that configuration bugs are a major cause of system failures and are notoriously difficult to detect due to their contextual and cross-cutting nature, improving their identification is not only a technical challenge but a critical step toward more robust, maintainable, and reliable software systems.

OPEN SCIENCE MATERIAL

To support the principles of open science and ensure transparency and reproducibility, all materials used in this study, including datasets, code, and experimental configurations are published and accessible here:

Unlabelled Eclipse Dataset
Eclipse dataset with automatically generated labels
UVLHub Labelled dataset

BIBLIOGRAPHY

[1] Hafiza Anisa Ahmed, Narmeen Zakaria Bawany, and Jawwad Ahmed Shamsi. "CaPBug-A Framework for Automatic Bug Categorization and Prioritization Using NLP and Machine Learning Algorithms." In: *IEEE Access* 9 (2021), pp. 50496–50512. DOI: 10.1109/ACCESS.2021.3069248.

[2] David Benavides, Chico Sundermann, Kevin Feichtinger, José Galindo, Rick Rabiser, and Thomas Thüm. "UVL: Feature modelling with the Universal Variability Language." In: *Journal of Systems and Software* 225 (Jan. 2025), p. 112326. DOI: 10.1016/j.jss.2024.112326.

[3] Tom B. Brown et al. "Language models are few-shot learners." In: *Proceedings of the 34th International Conference on Neural Information Processing Systems*. NIPS '20. Vancouver, BC, Canada: Curran Associates Inc., 2020. ISBN: 9781713829546.

[4] Martin Juan José Bucher and Marco Martini. *Fine-Tuned 'Small' LLMs (Still) Significantly Outperform Zero-Shot Generative AI Models in Text Classification*. 2024. arXiv: 2406.08660 [cs.CL]. URL: https://arxiv.org/abs/2406.08660.

[5] Gemma Catolino, Fabio Palomba, Andy Zaidman, and Filomena Ferrucci. "Not all bugs are the same: Understanding, characterizing, and classifying bug types." In: *Journal of Systems and Software* 152 (2019), pp. 165–181. ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2019.03.002. URL: https://www.sciencedirect.com/science/article/pii/S0164121219300536.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423. URL: https://aclanthology.org/N19-1423/.

[7] Sascha El-Sharkawy, Adam Krafczyk, and Klaus Schmid. "An Empirical Study of Configuration Mismatches in Linux." In: *Proceedings of the 21st International Systems and Software Product Line Conference - Volume A*. SPLC '17. Sevilla, Spain: Association for Computing Machinery, 2017, 19–28. ISBN: 9781450352215. DOI: 10.1145/3106195.3106208. URL: https://doi.org/10.1145/3106195.3106208.

[8] Fan Fang, John Wu, Yanyan Li, Xin Ye, Wajdi Aljedaani, and Mohamed Wiem Mkaouer. "On the classification of bug reports to improve bug localization." In: *Soft Comput.* 25.11 (June 2021), 7307–7323. ISSN: 1432-7643. DOI: 10.1007/s00500-021-05689-2. URL: https://doi.org/10.1007/s00500-021-05689-2.

[9] Michael Gegick, Pete Rotella, and Tao Xie. "Identifying security bug reports via text mining: An industrial case study." In: *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. 2010, pp. 11–20. DOI: 10.1109/MSR.2010.5463340.

[10] Michael Gegick, Pete Rotella, and Tao Xie. "Identifying security bug reports via text mining: An industrial case study." In: *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. 2010, pp. 11–20. DOI: 10.1109/MSR.2010.5463340.

[11] Santiago González-Carvajal and Eduardo Garrido-Merchán. *Comparing BERT against traditional machine learning text classification*. May 2020. DOI: 10.47852/bonviewJCCE3202838.

[12] Alif Al Hasan, Subarna Saha, Mia Mohammad Imran, and Tarannum Shaila Zaman. *LLPut: Investigating Large Language Models for Bug Report-Based Input Generation*. 2025. arXiv: 2503.20578 [cs.SE]. URL: https://arxiv.org/abs/2503.20578.

[13] Ahmed E. Hassan. "The road ahead for Mining Software Repositories." In: *2008 Frontiers of Software Maintenance*. 2008, pp. 48–57. DOI: 10.1109/FOSM.2008.4659248.

[14] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. Online manuscript released January 12, 2025. 2025. URL: https://web.stanford.edu/~jurafsky/slp3/.

[15] Huzefa Kagdi, Michael L. Collard, and Jonathan I. Maletic. "A survey and taxonomy of approaches for mining software repositories in the context of software evolution." In: *Journal of Software Maintenance and Evolution: Research and Practice* 19.2 (2007), pp. 77–131. DOI: https://doi.org/10.1002/smr.344. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/smr.344. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/smr.344.

[16] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. "Language Models are Unsupervised Multitask Learners." In: *OpenAI* (2019). Accessed: 2024-11-15. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[17] David Romero-Organvidez, José A. Galindo, Chico Sundermann, Jose-Miguel Horcas, and David Benavides. "UVLHub: A feature model data repository using UVL and open science principles." In: *Journal of Systems and Software* 216 (2024), p. 112150. ISSN: 0164-1212. DOI: https://doi.org/10.1016/j.jss.2024.112150. URL: https://www.sciencedirect.com/science/article/pii/S016412122400195X.

[18] Mohammed SAYAGH, Noureddine Kerzazi, Bram Adams, and Fabio Petrillo. "Software Configuration Engineering in Practice Interviews, Survey, and Systematic Literature Review." In: *IEEE Transactions on Software Engineering* 46.6 (2020), pp. 646–673. ISSN: 1939-3520. DOI: 10.1109/TSE.2018.2867847.

[19] Norbert Siegmund, Nicolai Ruckel, and Janet Siegmund. "Dimensions of software configuration: on the configuration context in modern software development." In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2020. Virtual Event, USA: Association for Computing Machinery, 2020, 338–349. ISBN: 9781450370431. DOI: 10.1145/3368089.3409675. URL: https://doi.org/10.1145/3368089.3409675.

[20] Nadia Tabassum, Abdallah Namoun, Tahir Alyas, Ali Tufail, Muhammad Taqi, and Ki-Hyung Kim. "Classification of Bugs in Cloud Computing Applications Using Machine Learning Techniques." In: *Applied Sciences* 13.5 (2023). ISSN: 2076-3417. DOI: 10.3390/app13052880. URL: https://www.mdpi.com/2076-3417/13/5/2880.

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need." In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, 6000–6010. ISBN: 9781510860964.

[22] Cathrin Weiss, Rahul Premraj, Thomas Zimmermann, and Andreas Zeller. "How Long Will It Take to Fix This Bug?" In: *Fourth International Workshop on Mining Software Repositories (MSR'07:ICSE Workshops 2007)*. 2007, pp. 1–1. DOI: 10.1109/MSR.2007.13.

[23] Wei Wen, Tingting Yu, and Jane Huffman Hayes. "CoLUA: Automatically Predicting Configuration Bug Reports and Extracting Configuration Options." In: *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*. 2016, pp. 150–161. DOI: 10.1109/ISSRE.2016.29.

[24] J. White, D.C. Schmidt, D. Benavides, P. Trinidad, and A. Ruiz–Cortés. "Automated Diagnosis of Product-Line Configuration Errors in Feature Models." In: *2008 12th International Software Product Line Conference*. 2008, pp. 225–234. DOI: 10.1109/SPLC.2008.16.

[25] Xin Xia, David Lo, Weiwei Qiu, Xingen Wang, and Bo Zhou. "Automated Configuration Bug Report Prediction Using Text Mining." In: *2014 IEEE 38th Annual Computer Software and Applications Conference*. 2014, pp. 107–116. DOI: 10.1109/COMPSAC.2014.17.

[26] Bowen Xu, David Lo, Xin Xia, Ashish Sureka, and Shanping Li. "EFSPredictor: Predicting Configuration Bugs with Ensemble Feature Selection." In: *2015 Asia-Pacific Software Engineering Conference (APSEC)*. 2015, pp. 206–213. DOI: 10.1109/APSEC.2015.38.

[27] Zuoning Yin, Xiao Ma, Jing Zheng, Yuanyuan Zhou, Lakshmi N. Bairavasundaram, and Shankar Pasupathy. "An empirical study on configuration errors in commercial and open source systems." In: *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*. SOSP '11. Cascais, Portugal: Association for Computing Machinery, 2011, 159–172. ISBN: 9781450309776. DOI: 10.1145/2043556.2043572. URL: https://doi.org/10.1145/2043556.2043572.

DECLARATION

I, the undersigned, **Noelia López Durán**, hereby declare that the present Master's Dissertation, entitled:

> *Configuration bugs classification using LLMs and encoders*: *a practical approach*

has been carried out independently and constitutes my own original work. I acknowledge that submitting work produced by another person, or reproducing texts, images, figures, or other content without proper attribution, constitutes plagiarism.

All sources consulted during the development of this work have been appropriately referenced and cited in the dissertation.

Sevilla, June 2025

Noelia López Durán